README — Analyseur statique Java

HAI913I — TP 1 : Analyse statique

SAMBARREY Adrien — PREVOST-AVENEL Pierre

Année universitaire 2026–2027 — Enseignant : Bachar Rima

1 Projet

Analyseur statique de programmes Java en ligne de commande. Il extrait la structure et les appels de méthodes avec le type statique du receveur, construit un modèle de faits indépendant de la bibliothèque d'analyse, puis calcule treize métriques et un graphe d'appel interrogeable dans les deux sens. Bonus traités : extraction Spoon (C1), export Graphviz et interface graphique (C2).

Structure du code

```
hai913i.tp1
|-- Main          point d'entree : lit les arguments, delegue
|-- cli           Action, Options, CommandLine, Analyzer
|-- parse         ProjectSources, JdtParser            <- connait JDT
|-- extract       AstPrinter, StructureVisitor,
|                 CallCollector, FactExtractor         <- connait JDT
|-- model         TypeFact, FieldFact, MethodFact,
|                 CallFact, ProjectFacts               <- ne depend de
   rien
|-- metrics       Ranking, MetricsService, MetricsReport
|-- callgraph     CallGraph, CallGraphBuilder
|-- report        TextReport, DotReport
|-- ui            AnalyzerWindow                (bonus C2)
'-- spoon         SpoonExtractor, SpoonRawCounts (bonus C1)
```

Entrée : un dossier de projet Java. La racine des sources est détectée automatiquement (`src/main/java`, sinon `src`, sinon le dossier du projet) ; `target`, `build`, `out`, `bin` et les dossiers cachés sont ignorés.

Sorties : texte sur la sortie standard, export DOT, interface graphique. Erreurs de compilation et messages d'erreur sur la sortie d'erreur.

Limites : pas d'expansion des appels virtuels (ni CHA ni RTA), un appel via une interface visant la méthode de l'interface ; réflexion, proxys et injection de dépendances non vus ; pas d'arc vers les constructeurs, qui restent des nœuds.

2 Environnement

Élément	Version exacte utilisée
JDK	OpenJDK 17.0.16 (Homebrew, aarch64). **Java 17 minimum**
Maven	3.9.11
Eclipse JDT Core	`org.eclipse.jdt:org.eclipse.jdt.core:3.46.0`
Spoon (bonus C1)	`fr.inria.gforge.spoon:spoon-core:11.5.0`
SLF4J (bonus C1)	`org.slf4j:slf4j-nop:2.0.18`
JUnit Jupiter	5.14.4
Graphviz (bonus C2)	14.1.0, commande `dot`
Système d'exploitation	macOS 26.6, Apple Silicon
IDE	IntelliJ IDEA 2026.2

Aucun JAR n'est à installer à la main : Maven télécharge toutes les dépendances. Le projet est sur le *classpath*, sans `module-info.java`.

Attention : `JAVA_HOME` et la commande `java` doivent désigner le même JDK 17. Une incohérence produit un `UnsupportedClassVersionError` au lancement du JAR.

3 Construction

Depuis une copie fraîche du dépôt, sans IDE :

```
cd hai913i-tp1-analyzer
mvn clean package
```

Produit `target/hai913i-tp1-analyzer.jar`, exécutable et contenant ses dépendances.

Pour lancer la suite de tests seule :

```
mvn clean test
```

4 Exécution

Syntaxe

```
java -jar target/hai913i-tp1-analyzer.jar PROJET [OPTIONS]

  PROJET                   dossier du projet Java a analyser
                           facultatif avec --gui

Options :
  --action ACTION          ast | structure | calls | metrics | graph | spoon
      | all
                           (defaut : all)
  --x N                    seuil X, entier positif ou nul
                           obligatoire pour les actions metrics et all
  --callees ID             methodes appelees par ID
  --callers ID             methodes appelantes de ID
                           ID a la forme paquetage.Classe#methode(Type,Type)
  --dot FICHIER            exporte le graphe au format Graphviz
  --dot-package PREFIXE    limite l'export DOT a un paquetage
  --file NOM               avec --action ast : n'affiche que ce fichier
  --gui                    ouvre l'interface graphique
```

Exemples complets

```
# Analyse complete du projet de validation, seuil X = 5
java -jar target/hai913i-tp1-analyzer.jar resources/validation --x 5

# Arbre syntaxique d'un fichier (etape A1)
java -jar target/hai913i-tp1-analyzer.jar resources/validation --action
    ast --file Dvd.java

# Graphe d'appel et interrogation dans les deux sens
java -jar target/hai913i-tp1-analyzer.jar resources/validation --action
    graph \
     --callers 'library.model.Loanable#checkOut(library.model.Member)'

# Export Graphviz, puis image
java -jar target/hai913i-tp1-analyzer.jar resources/validation --action
    graph \
     --dot graphe.dot --dot-package library.service
dot -Tpng graphe.dot -o graphe.png

# Comparaison JDT / Spoon (bonus C1)
java -jar target/hai913i-tp1-analyzer.jar resources/validation --action
    spoon

# Interface graphique (bonus C2)
java -jar target/hai913i-tp1-analyzer.jar --gui
```

Sous Windows, entourez de guillemets les chemins contenant des espaces.

Codes de sortie

Code	Signification
0	analyse réussie
2	usage incorrect : argument manquant, option ou action inconnue, X mal formé ou absent
3	chemin invalide ou illisible
4	aucun fichier `.java` trouvé

Le projet est vérifié avant le seuil : un chemin inexistant rend 3 et un dossier sans source rend 4, même si `--x` a été oublié. Le code 2 pour seuil absent ne se produit donc que sur un projet par ailleurs valide.

5 Validation

Projet d'entrée

Le projet de validation fourni avec le sujet. **Il est livré avec le dépôt**, sous `resources/validation`, ainsi que les trois échantillons de robustesse sous `resources/robustness` : `mvn clean test` réussit donc sur une copie fraîche, sans avoir à récupérer les ressources du sujet séparément.

L'outil a également été exécuté sur deux projets extérieurs, pour vérifier que les résultats ne dépendent pas du projet de validation :

— le paquetage `gof` de `resources/visitor-examples`, recopié dans le `src` d'un dossier vide (9 fichiers, 0 erreur, 9 types, 19 méthodes, 40 appels, graphe de 19 nœuds et 14 arcs) ; il contient des *records*, absents du projet de validation ;

— l'analyseur lui-même (bonus C3 : 29 fichiers, 1 996 lignes, 10 paquetages, 130 méthodes, 1 066 appels). Ses dépendances JDT et Spoon n'étant pas sur le *classpath* d'analyse, 247 appels sont signalés non résolus ; l'analyse va jusqu'au bout.

Le projet de validation recopié dans un dossier dont le nom contient des espaces (`mon projet test`) rend les mêmes chiffres, sans guillemets particuliers sous macOS.

Sortie attendue

```
Racine des sources     : .resources/validation/src
Unites de compilation  : 14
Erreurs de compilation : 0

Types      : 15
Methodes   : 59 (dont 9 constructeurs)
Attributs  : 21
Paquetages : 4 [library.app, library.model, library.service, library.util]
Appels     : 94 (46 internes, 48 externes, 0 non resolus)

== Metriques ==      (avec --x 5)
 1. Classes : 15   2. Lignes : 493   3. Methodes : 59   4. Paquetages : 4
 5. Methodes/classe : 3.93   6. Lignes/methode : 4.64 (246 pour 53)
 7. Attributs/classe : 1.40  13. Max parametres : 5 (Statistics#report)

== Graphe d'appel ==
Noeuds : 59 | Arcs : 44 | Sites d'appel internes : 46
Appels externes : 48 | Appels non resolus : 0
```

Vérification automatique

La commande suivante rejoue l'intégralité des points de contrôle A0 à B4, plus l'équivalence entre l'extraction JDT et l'extraction Spoon :

```
mvn clean test
```

```
Tests run: 37, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Comportement sur les échantillons de robustesse

Échantillon	Comportement attendu
`missing-dep`	2 erreurs signalées ; `StringUtils.capitalize` non résolu, `formatter.decorate` résolu ; l'analyse va jusqu'au bout ; code 0
`syntax-error`	erreurs de `Broken.java` signalées, `Helper.java` analysé normalement, l'outil ne s'arrête pas ; code 0
`no-java-files`	message « aucun fichier .java sous ... » ; code 4, que `--x` soit fourni ou non
chemin inexistant	message « Pas un repertoire : ... » ; code 3, que `--x` soit fourni ou non
`--x abc`	message « seuil X mal forme » et rappel de l'usage ; code 2
`--x` absent	sur un projet valide : message « le seuil X est obligatoire » et usage ; code 2

6 Dépannage

Symptôme	Cause et remède
`UnsupportedClassVersionError` au lancement	Le JDK d'exécution est antérieur à 17. Vérifier `java -version` et `JAVA_HOME` ; les deux doivent désigner le même JDK 17.
`SecurityException: Invalid signature file digest`	Les JAR Eclipse sont signés. Le `pom.xml` exclut déjà `META-INF/*.SF`, `*.DSA`, `*.RSA`, `*.EC` : reconstruire avec `mvn clean package`.
`IllegalStateException: Missing system library`	`setEnvironment` n'inclut pas le JDK courant. Le 4e argument doit valoir `true`.
Environ la moitié des appels non résolus, **sans aucune erreur**	La racine des sources désigne le dossier du projet au lieu du dossier qui contient les paquetages (`src`). Panne silencieuse : vérifier la ligne « Racine des sources » affichée.
14 types au lieu de 15	Seul `visit(TypeDeclaration)` est redéfini. Les énumérations et les *records* sont des nœuds distincts.
93 appels au lieu de 94	`SuperMethodInvocation` oublié : `super.m()` n'est pas un `MethodInvocation`.
`cannot find symbol: method of(String)` sur `Path`	Import automatique erroné : `org.eclipse.core.runtime.Path` au lieu de `java.nio.file.Path`.
L'IDE affiche des erreurs mais `mvn` compile	Modèle de projet de l'IDE périmé ou SDK mal configuré. Recharger le projet Maven ; la ligne de commande fait foi.
Spoon affiche un avertissement SLF4J	Ajouter `org.slf4j:slf4j-nop` (déjà présent dans le `pom.xml`).
L'image DOT est illisible	Filtrer avec `--dot-package`, par exemple `library.service`.

7 Dépôt et vidéo

— Dépôt Git : `https://github.com/AdrienSmbr/TP1-HAI913I`
— Vidéo de démonstration : `https://youtu.be/UB8gPc2GF-g`